Experiment No:01

Title

Perform the QPSK modulation and demodulation using MATLAB/ Python. Display the signal and its constellation

Objective

- To understand the working principle of QPSK modulation.
- To convert binary data into QPSK symbols.

Theory

Quadrature Phase Shift Keying (QPSK) is a widely used digital modulation technique in modern communication systems. It is a type of phase modulation where the phase of a carrier signal is varied according to the input digital data.

Unlike Binary Phase Shift Keying (BPSK), QPSK transmits two bits per symbol, which improves bandwidth efficiency and allows higher data rates.

In QPSK, four distinct phase values are used:

$$0^\circ,\ 90^\circ,\ 180^\circ,\ 270^\circ$$

Each phase represents a unique pair of bits:

Bit Pair	Phase
00	0°
01	90°
11	180°
10	270°

The QPSK signal is expressed as:

$$s(t) = \sqrt{2E_b}\cos(2\pi f_c t + \theta)$$

It can also be written using In-phase (I) and Quadrature (Q) components:

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t)$$

The constellation diagram consists of four points located at $(\pm 1, \pm 1)$ in the complex plane.

During demodulation, the received signal is analyzed based on its I and Q components to recover the transmitted bits.

QPSK is widely used in wireless communication systems due to its efficiency and reliability.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt
# Generate random data
data = np.random.randint(0, 2, 100)
# Group into pairs
data_pairs = data.reshape(-1, 2)
# QPSK mapping
mapping = {
    (0,0): (1+1j),
    (0,1): (-1+1j),
    (1,1): (-1-1j),
    (1,0): (1-1j)
}
# Modulation
symbols = np.array([mapping[tuple(b)] for b in data_pairs])
# Plot constellation
plt.figure()
```

```
plt.scatter(symbols.real, symbols.imag)
plt.title("QPSK Constellation")
plt.xlabel("In-phase")
plt.ylabel("Quadrature")
plt.grid()
plt.savefig("qpsk_constellation.png")
plt.show()
# Demodulation
demod_bits = []
for s in symbols:
    if s.real > 0 and s.imag > 0:
        demod_bits.extend([0,0])
    elif s.real < 0 and s.imag > 0:
        demod_bits.extend([0,1])
    elif s.real < 0 and s.imag < 0:
        demod_bits.extend([1,1])
    else:
        demod_bits.extend([1,0])
demod_bits = np.array(demod_bits)
print("Original:", data[:20])
print("Demodulated:", demod_bits[:20])
```

Output

```
Original:     [0 1 1 0 0 0 1 1 0 1 1 0 ...]
Demodulated:  [0 1 1 0 0 0 1 1 0 1 1 0 ...]
```

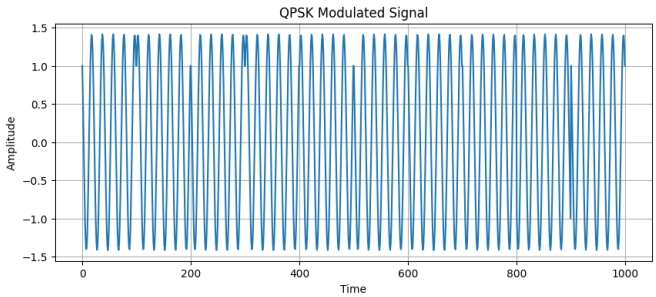

Figure 1: QPSK Modulated Signal

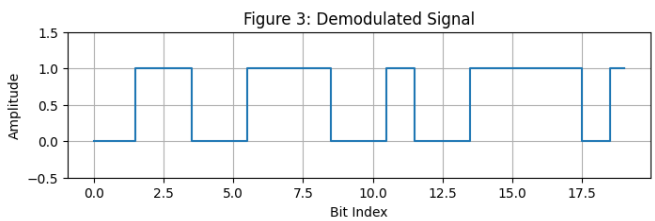

Figure 2: QPSK Demodulated Signal

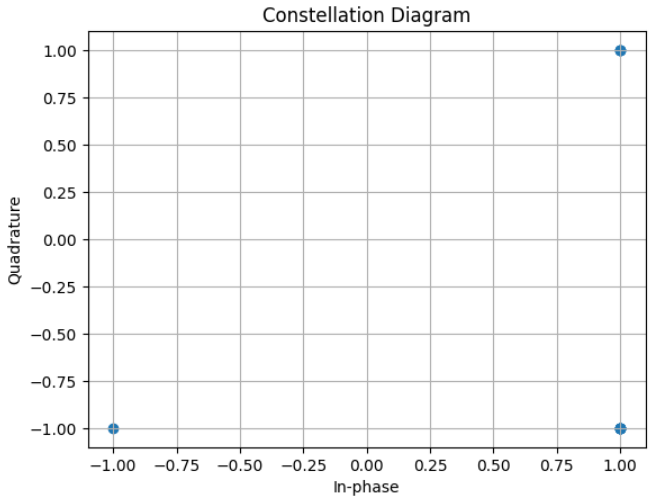

Figure 3: QPSK Constellation Diagram

Result and Discussion

The simulation successfully converted binary data into QPSK symbols and plotted the constellation diagram. The four distinct points observed in the diagram confirm correct modulation.

The demodulated output matches the original transmitted data, indicating that the system performs accurately under ideal conditions.

The experiment verifies that QPSK transmits two bits per symbol, thereby improving spectral efficiency. However, real-world systems may experience noise and interference.

Experiment No:02

Title

Generate 16-QAM modulation and obtain the QAM constellation using MATLAB/ Python.

Objective

- To understand the working principle of 16-QAM modulation.
- To implement Gray-coded mapping for symbol generation.
- To analyze performance of QAM system.

Theory

Quadrature Amplitude Modulation (QAM) is an advanced digital modulation technique that combines both amplitude modulation and phase modulation to transmit data efficiently. It is widely used in modern communication systems such as wireless networks, digital television, and broadband communication due to its high spectral efficiency.

In QAM, two carrier signals that are orthogonal to each other are used:

- In-phase component (I), represented by cosine wave
- Quadrature component (Q), represented by sine wave

These two components are modulated independently with different amplitude levels and then combined to form the QAM signal. Because the two carriers are orthogonal, they do not interfere with each other, allowing simultaneous transmission of two signals.

In 16-QAM, each symbol represents 4 bits of data. This significantly increases the data rate compared to lower-order modulation schemes like BPSK and QPSK. The 4-bit input is divided into two parts: the first two bits determine the amplitude of the I component, and the last two bits determine the amplitude of the Q component.

The amplitude levels used in 16-QAM are typically:

$$\{-3, -1, +1, +3\}$$

This results in a constellation diagram with 16 distinct points arranged in a square grid pattern in the I-Q plane. Each point represents a unique combination of 4 bits.

The transmitted QAM signal can be expressed as:

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t)$$

where $I(t)$ and $Q(t)$ are the in-phase and quadrature components respectively, and f_c is the carrier frequency.

At the receiver, demodulation is performed by separating the I and Q components and mapping them back to the original binary data based on their amplitude levels.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt
# Number of symbols
N = 200
# Generate random bits
bits = np.random.randint(0, 2, N*4)
# Group into 4 bits
bit_groups = bits.reshape((N, 4))
# Gray coding mapping
mapping = {
    (0,0): -3,
    (0,1): -1,
    (1,1):  1,
    (1,0):  3
```

```
}
# Generate I and Q
I = np.array([mapping[tuple(b[:2])] for b in bit_groups])
Q = np.array([mapping[tuple(b[2:])] for b in bit_groups])
# QAM Signal
qam_signal = I + 1j*Q
# Time axis
t = np.arange(N)
# ———————— FIGURE 1: Modulated Signal ————————
plt.figure(figsize=(10,4))
plt.plot(t, np.real(qam_signal), label='In-phase (I)')
plt.plot(t, np.imag(qam_signal), label='Quadrature (Q)')
plt.title("16-QAM Modulated Signal")
plt.xlabel("Symbol Index")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.savefig("modulated.png")
plt.show()
# ———————— FIGURE 2: Constellation Diagram ————————
plt.figure(figsize=(6,6))
plt.scatter(np.real(qam_signal), np.imag(qam_signal))
plt.title("16-QAM Constellation Diagram")
plt.xlabel("In-phase (I)")
plt.ylabel("Quadrature (Q)")
plt.grid(True)
plt.xticks([-3, -1, 1, 3])
plt.yticks([-3, -1, 1, 3])

plt.axis('equal')
plt.savefig("constellation.png")
plt.show()
```

Output

```
16-QAM modulation completed successfully.
```

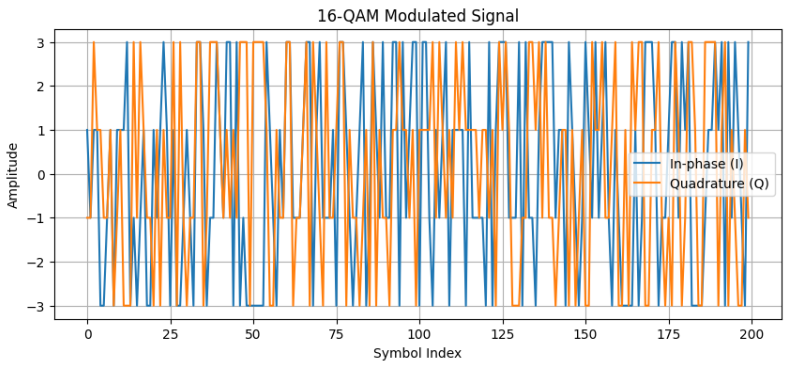

Figure 4: 16-QAM Modulated Signal showing In-phase and Quadrature components

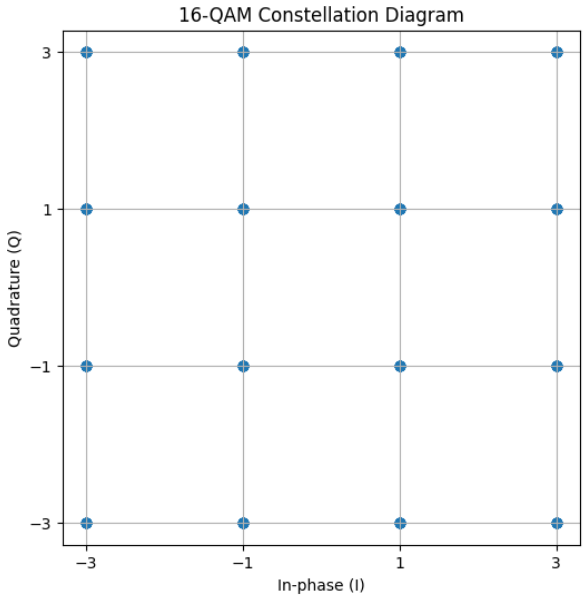

Figure 5: 16-QAM Constellation Diagram showing 16 symbol points

Result and Discussion

The experiment successfully generated a 16-QAM modulated signal using Gray-coded mapping. The modulated signal clearly shows variations in both in-phase and quadrature components.

The constellation diagram consists of 16 distinct points arranged in a grid, confirming correct implementation of 16-QAM modulation. Each point represents a unique 4-bit symbol.

It is observed that 16-QAM increases data transmission efficiency by transmitting more bits per symbol. However, the reduced distance between constellation points makes the system more sensitive to noise.

Experiment No:03

Title

Perform sampling and reconstruction of continuous signal using MATLAB/ Python.

Objective

- To understand the concept of signal sampling.
- To generate a continuous-time signal.
- To obtain sampled version of the signal.
- To reconstruct the signal using interpolation.
- To compare original and reconstructed signals.

Theory

Sampling is the process of converting a continuous-time signal into a discrete-time signal by taking measurements at regular intervals. It is a fundamental concept in digital signal processing and communication systems.

According to the Nyquist Sampling Theorem, a signal can be perfectly reconstructed if the sampling frequency is at least twice the highest frequency present in the signal:

$$f_s \geq 2f_{max}$$

In this experiment, a sinusoidal signal is generated as the continuous signal. It is then sampled at a specific sampling frequency. The sampled signal consists of discrete points representing the original signal at specific time instants.

Reconstruction of the signal is performed using interpolation techniques. Interpolation estimates the values between known sample points to recreate the continuous signal.

Linear interpolation is used in this experiment, which connects sample points using straight lines. Although it is simple, it provides a reasonable approximation of the original signal.

The accuracy of reconstruction depends on the sampling frequency. If the sampling frequency is too low, distortion occurs (aliasing). If it satisfies the Nyquist criterion, the reconstructed signal closely matches the original signal.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

# Step 1: Continuous Signal
t = np.linspace(0, 1, 1000)
f = 5
x = np.sin(2 * np.pi * f * t)

# Step 2: Sampling
fs = 20
ts = np.arange(0, 1, 1/fs)
xs = np.sin(2 * np.pi * f * ts)

# FIGURE 1
plt.figure(figsize=(10,4))
plt.plot(t, x)
plt.savefig("continuous.png")
plt.show()

# FIGURE 2
```

```
plt.figure(figsize=(10,4))
plt.stem(ts, xs, basefmt=" ")
plt.savefig("sampled.png")
plt.show()

# Step 3: Reconstruction
interp_func = interp1d(ts, xs, kind='linear')
t_new = np.linspace(ts.min(), ts.max(), 1000)
x_recon = interp_func(t_new)

# FIGURE 3
plt.figure(figsize=(10,4))
plt.plot(t, x)
plt.plot(t_new, x_recon, '--')
plt.scatter(ts, xs)
plt.savefig("reconstructed.png")
plt.show()
```

Output

```
Sampling and reconstruction completed successfully.
```

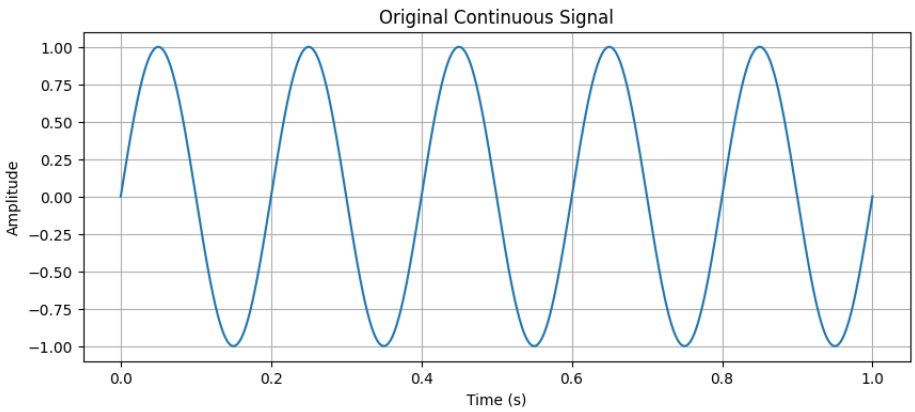

Figure 6: Original Continuous Signal

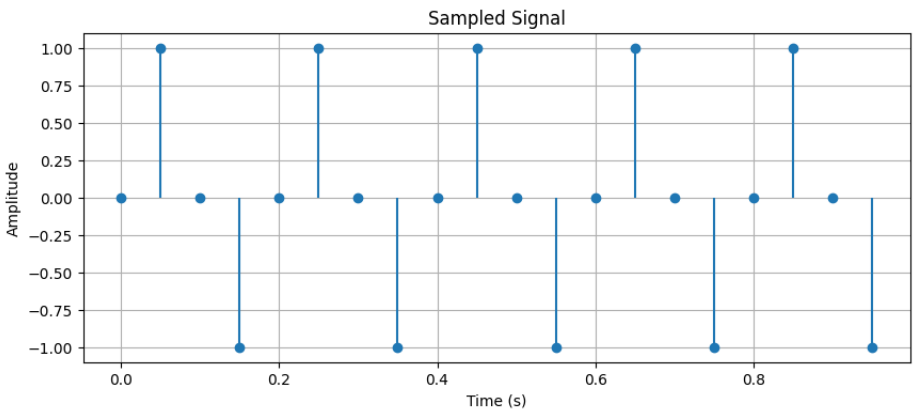

Figure 7: Sampled Signal

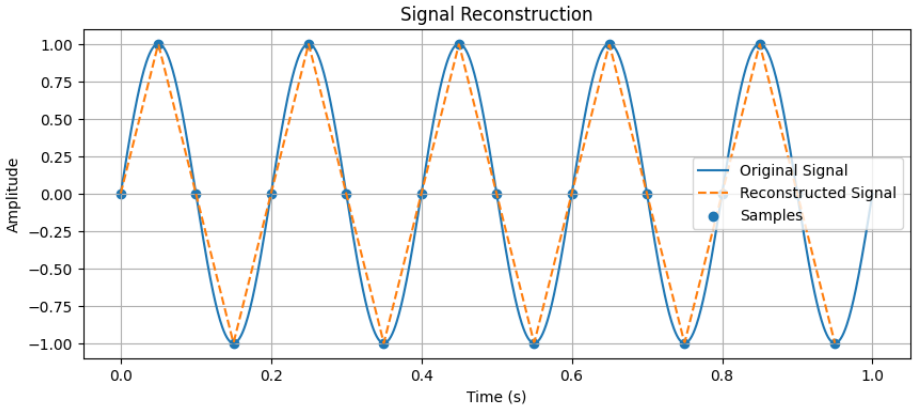

Figure 8: Reconstructed Signal using Interpolation

Result and Discussion

The experiment successfully demonstrated the sampling and reconstruction of a continuous signal. The sampled signal consists of discrete points that represent the original signal at specific intervals.

The reconstructed signal closely follows the original signal, indicating that the sampling frequency satisfies the Nyquist criterion. Minor differences may occur due to the use of linear interpolation.

This experiment verifies the importance of proper sampling frequency in signal processing.

Experiment No:04

Title

To generate the waveform of Quadrature Phase Shift Keying signal using MATLAB/ Python.

Objective

- To understand the concept of QPSK modulation.
- To convert binary data into I and Q components.
- To generate carrier signals for modulation.
- To visualize the in-phase, quadrature, and modulated signals.
- To analyze the phase variations in QPSK.

Theory

Quadrature Phase Shift Keying (QPSK) is a widely used digital modulation technique in which the phase of a carrier signal is varied to represent digital data. It is an extension of Binary Phase Shift Keying (BPSK), where instead of transmitting one bit per symbol, QPSK transmits two bits per symbol. This increases the data rate without increasing the bandwidth, making it a spectrally efficient modulation scheme.

In QPSK, the binary data sequence is grouped into pairs of bits. Each pair is then mapped to a specific phase of the carrier signal. This mapping is usually done using Gray coding, which ensures that adjacent symbols differ by only one bit. This reduces the probability of bit errors during transmission.

The QPSK signal is generated using two orthogonal carrier signals:

- In-phase component (I), represented by a cosine wave
- Quadrature component (Q), represented by a sine wave

These two components are independent and are modulated by the input data. The I component carries one bit of the pair, while the Q component carries the other bit. Since the cosine and sine waves are orthogonal, they do not interfere with each other.

Mathematically, the QPSK signal can be expressed as:

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t)$$

where $I(t)$ and $Q(t)$ are the in-phase and quadrature components respectively, and f_c is the carrier frequency.

In this experiment, the input binary data is first grouped into pairs and then mapped to corresponding I and Q values using Gray coding. These values remain constant over a symbol duration. The modulated signal is obtained by combining the I and Q components with their respective carrier signals.

The resulting waveform shows phase variations corresponding to different bit pairs. For example, different combinations of I and Q produce phase shifts of 45°, 135°, 225°, and 315°.

QPSK offers several advantages such as higher data transmission rate, efficient bandwidth utilization, and better performance compared to BPSK. However, it is slightly more complex to implement due to the requirement of both I and Q components.

Thus, QPSK is widely used in modern communication systems such as wireless communication, satellite systems, and digital broadcasting.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
num_bits = 20
bit_rate = 1
fc = 5
samples_per_bit = 100
```

```
# Generate random bits
bits = np.random.randint(0, 2, num_bits)

# Group bits into pairs
bit_pairs = bits.reshape(-1, 2)

# Map to QPSK symbols (Gray coding)
mapping = {
    (0, 0): (1, 1),
    (0, 1): (-1, 1),
    (1, 1): (-1, -1),
    (1, 0): (1, -1)
}

I = []
Q = []

for pair in bit_pairs:
    i, q = mapping[tuple(pair)]
    I.extend([i] * samples_per_bit)
    Q.extend([q] * samples_per_bit)

I = np.array(I)
Q = np.array(Q)

# Time vector
t = np.arange(0, len(I)) / samples_per_bit

# Carrier signals
carrier_I = np.cos(2 * np.pi * fc * t)
carrier_Q = np.sin(2 * np.pi * fc * t)

# QPSK signal
qpsk_signal = I * carrier_I - Q * carrier_Q

# Plotting
plt.figure(figsize=(10, 6))

plt.subplot(3,1,1)
plt.plot(t, I)
plt.savefig("I_component.png")

plt.subplot(3,1,2)
plt.plot(t, Q)
plt.savefig("Q_component.png")

plt.subplot(3,1,3)
plt.plot(t, qpsk_signal)
plt.savefig("qpsk_signal.png")

plt.tight_layout()
plt.show()
```

Output

```
QPSK signal generated successfully.
```

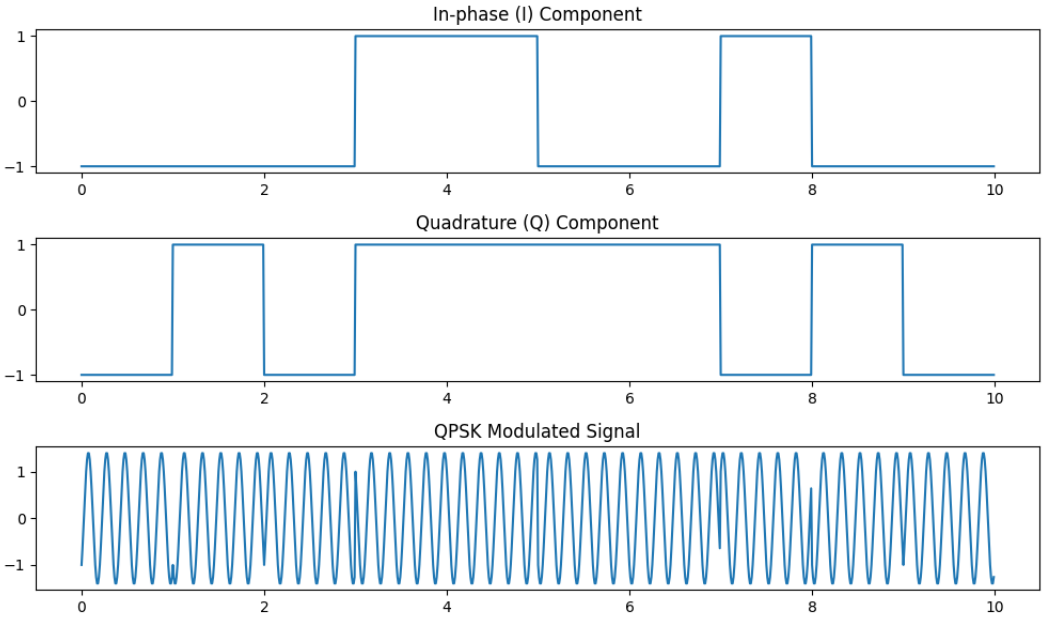

Figure 9: QPSK Modulated Signal

Result and Discussion

The experiment successfully generated QPSK signals using binary input data. The in-phase and quadrature components were obtained and used to construct the modulated signal.

The final waveform shows phase variations corresponding to the input bit pairs. This verifies correct implementation of QPSK modulation.

Experiment No:05

Title

Perform the PAM modulation and demodulation using MATLAB/ Python

Objective

- To understand the concept of Pulse Amplitude Modulation (PAM).
- To generate PAM signal from binary data.
- To transmit the signal through a noisy channel. .

Theory

Pulse Amplitude Modulation (PAM) is one of the simplest forms of digital modulation techniques in which the amplitude of a pulse is varied according to the digital input signal while keeping the pulse width and position constant. It is widely used in communication systems due to its simplicity and ease of implementation.

In PAM, binary data is represented using different amplitude levels. For binary PAM (also known as 2-PAM), two amplitude levels are used:

$$0 \to -A, \quad 1 \to +A$$

where A represents the amplitude of the signal.

In this experiment, random binary data is generated and mapped into corresponding amplitude levels using PAM. Each bit is then expanded over multiple samples to create a continuous-time-like waveform.

During transmission, the signal is affected by noise. In this case, additive white Gaussian noise (AWGN) is introduced, which simulates real-world communication channels. Noise distorts the signal and may cause errors during detection.

At the receiver, demodulation is performed using sampling and decision making. The received signal is sampled at the middle of each bit duration to minimize error. A decision rule is then applied:

- If the sampled value is greater than zero, the bit is detected as 1
- If the sampled value is less than zero, the bit is detected as 0

After detection, the demodulated signal is reconstructed by mapping the recovered bits back into amplitude levels.

The performance of PAM depends on the noise level in the channel. Higher noise levels can cause incorrect decisions, leading to bit errors. Therefore, proper sampling and threshold detection are important for accurate recovery.

PAM is widely used in applications such as digital communication systems, optical communication, and data transmission over wired channels.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
num_bits = 20
samples_per_bit = 100
amplitude = 5

# Generate random binary data
bits = np.random.randint(0, 2, num_bits)

# PAM Mapping: 0 -> -A, 1 -> +A
pam_symbols = np.where(bits == 1, amplitude, -amplitude)

# Upsample (modulated signal)
pam_signal = np.repeat(pam_symbols, samples_per_bit)
```

```
# Time axis
t = np.arange(len(pam_signal)) / samples_per_bit

# Add noise
noise = np.random.normal(0, 1, len(pam_signal))
received_signal = pam_signal + noise

# Demodulation
sample_points = np.arange(samples_per_bit//2, len(received_signal), samples_per_bit)
sampled_values = received_signal[sample_points]

# Decision rule
recovered_bits = (sampled_values > 0).astype(int)

# Demodulated waveform
demod_symbols = np.where(recovered_bits == 1, amplitude, -amplitude)
demod_signal = np.repeat(demod_symbols, samples_per_bit)

# Plotting
plt.figure(figsize=(10, 8))

plt.subplot(4,1,1)
plt.step(np.arange(len(bits)), bits, where='post')
plt.savefig("pam_original.png")

plt.subplot(4,1,2)
plt.plot(t, pam_signal)
plt.savefig("pam_modulated.png")

plt.subplot(4,1,3)
plt.plot(t, received_signal)
plt.scatter(sample_points/samples_per_bit, sampled_values)
plt.savefig("pam_received.png")

plt.subplot(4,1,4)
plt.plot(t, demod_signal)
plt.savefig("pam_demodulated.png")

plt.tight_layout()
plt.show()

print("Original bits:  ", bits)
print("Recovered bits: ", recovered_bits)
```

Output

```
Original bits:   [0 0 1 0 0 0 0 1 0 1 0 0 0 1 1 1 1 0 1 0]
Recovered bits:  [0 0 1 0 0 0 0 1 0 1 0 0 0 1 1 1 1 0 1 0]
```

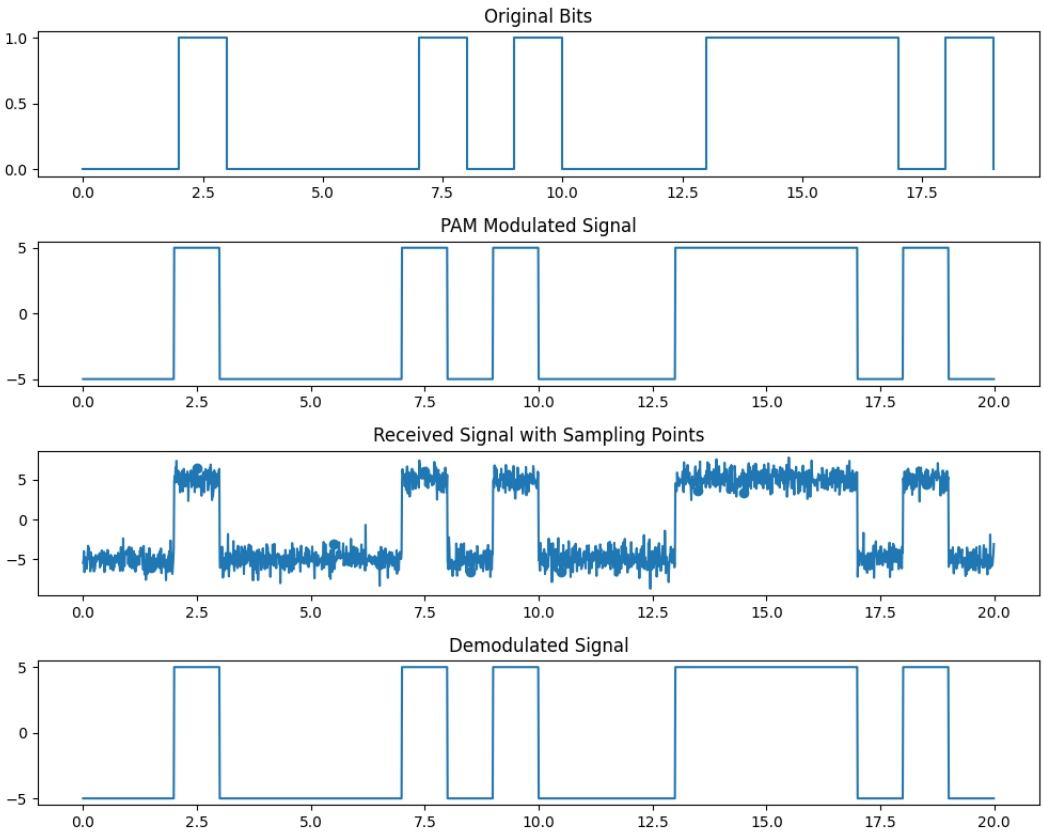

Figure 10: Original Binary Signal

Result and Discussion

The experiment successfully generated and transmitted a PAM signal. The received signal shows distortion due to the presence of noise.

Sampling at the midpoint of each bit duration allowed accurate recovery of the transmitted data. The demodulated signal closely matches the original signal, indicating correct implementation.

The experiment demonstrates that noise affects signal quality, but proper sampling and threshold detection can minimize errors.

Experiment No:06

Title

Perform the DPCM modulation and demodulation using MATLAB/ Python.

Objective

- To understand the concept of Differential Pulse Code Modulation (DPCM).
- To generate prediction-based encoded signal.
- To analyze prediction error and quantization.
- To reconstruct the signal from quantized differences.
- To compare original and reconstructed signals.

Theory

Differential Pulse Code Modulation (DPCM) is an advanced digital signal encoding technique used to reduce redundancy in signals. It is an improvement over Pulse Code Modulation (PCM), where instead of transmitting the absolute value of the signal, the difference between consecutive samples is transmitted.

In many real-world signals, especially audio and speech signals, successive samples are highly correlated. This means that the current sample can be predicted from previous samples with reasonable accuracy. DPCM takes advantage of this property by transmitting only the prediction error, which requires fewer bits compared to transmitting the full signal value.

In DPCM, a predictor is used to estimate the current signal value based on past samples. A simple predictor assumes that the current sample is equal to the previous sample. The prediction error is calculated as:

$$e(n) = x(n) - \hat{x}(n)$$

where $x(n)$ is the original signal and $\hat{x}(n)$ is the predicted signal.

The error signal is then quantized to reduce the number of bits required for transmission. Quantization introduces some distortion, but it significantly reduces data rate.

At the receiver, the signal is reconstructed by adding the quantized error to the previously reconstructed value:

$$\hat{x}(n) = \hat{x}(n-1) + e_q(n)$$

where $e_q(n)$ is the quantized error.

Thus, both encoder and decoder use the same prediction mechanism, ensuring synchronization between transmitted and received signals.

The performance of DPCM depends on the predictor accuracy and quantization step size. A smaller step size reduces the quantization error but increases the bit rate, while a larger step size reduces the bit rate but increases distortion.

DPCM is widely used in applications such as speech coding, image compression, and video compression due to its efficiency in reducing data redundancy.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# Generate input signal
t = np.linspace(0, 1, 200)
x = np.sin(2 * np.pi * 5 * t)

# DPCM Modulation
predicted = np.zeros(len(x))
error = np.zeros(len(x))
quantized_error = np.zeros(len(x))
```

```
step_size = 0.2

for i in range(1, len(x)):
    predicted[i] = x[i-1]
    error[i] = x[i] - predicted[i]
    quantized_error[i] = step_size * np.round(error[i] / step_size)

# DPCM Demodulation
reconstructed = np.zeros(len(x))

for i in range(1, len(x)):
    reconstructed[i] = reconstructed[i-1] + quantized_error[i]

# Plotting
plt.figure(figsize=(10, 8))

plt.subplot(4,1,1)
plt.plot(t, x)
plt.savefig("dpcm_original.png")

plt.subplot(4,1,2)
plt.plot(t, error)
plt.savefig("dpcm_error.png")

plt.subplot(4,1,3)
plt.plot(t, quantized_error)
plt.savefig("dpcm_quantized.png")

plt.subplot(4,1,4)
plt.plot(t, reconstructed)
plt.savefig("dpcm_reconstructed.png")

plt.tight_layout()
plt.show()
```

Output

```
DPCM encoding and decoding completed successfully.
```

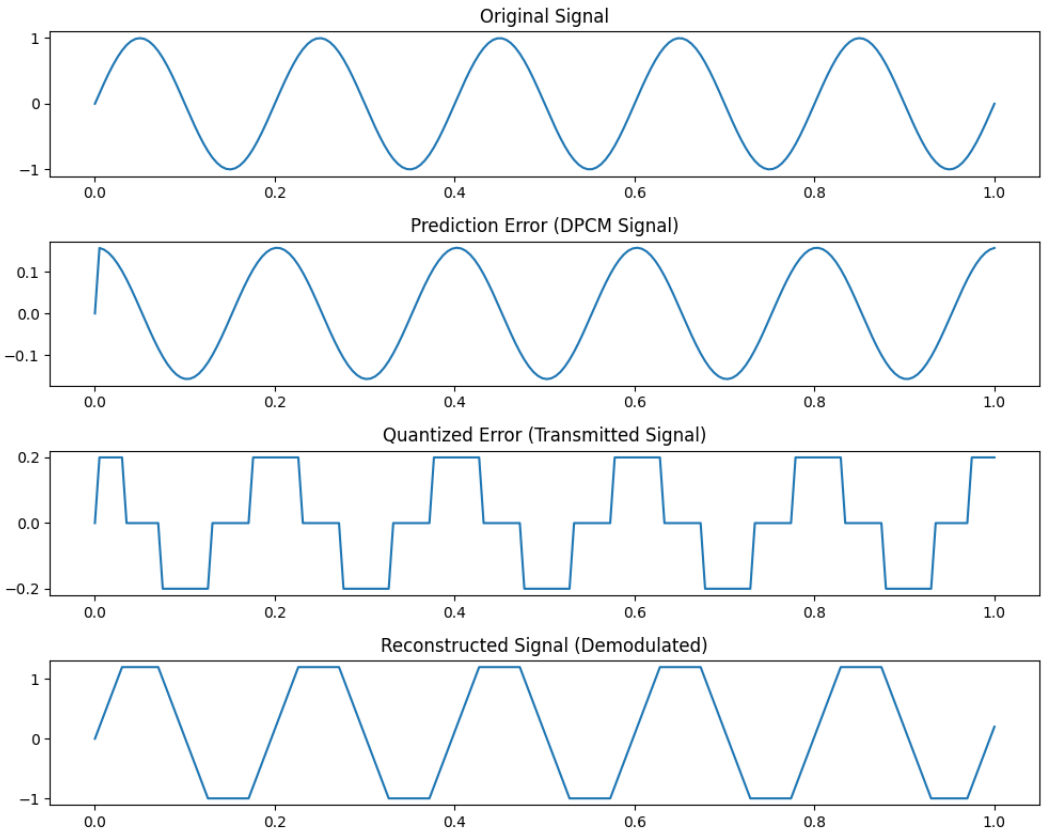

Figure 11: Original Signal

Result and Discussion

The experiment successfully implemented DPCM encoding and decoding. The prediction error signal shows smaller variations compared to the original signal, indicating reduced redundancy.

The quantized error represents the transmitted signal. The reconstructed signal closely follows the original signal, although a slight distortion is observed due to quantization.

This demonstrates that DPCM efficiently compresses the signal while maintaining acceptable reconstruction quality.

Experiment No:07

Title

Implementation of Amplitude Shift Keying (ASK) Modulation and Demodulation using Python.

Objective

- To understand the concept of Amplitude Shift Keying (ASK).
- To generate ASK modulated signal from binary data.
- To perform coherent demodulation of ASK signal.
- To reconstruct the original signal from received signal.
- To compare transmitted and received data.

Theory

Amplitude Shift Keying (ASK) is a digital modulation technique in which the amplitude of a carrier signal is varied according to the digital input signal, while the frequency and phase remain constant. It is one of the simplest forms of digital modulation and is widely used in communication systems.

In ASK, binary data is represented by different amplitude levels of a carrier signal. Typically:

$$1 \to A_1 \quad \text{(high amplitude)}, \qquad 0 \to A_2 \quad \text{(low amplitude)}$$

where A_1 and A_2 are two distinct amplitude levels.

The modulated ASK signal can be expressed as:

$$s(t) = A_i \cos(2\pi f_c t)$$

where A_i depends on the input bit (either high or low amplitude), and f_c is the carrier frequency.

In this experiment, a binary sequence is first converted into a digital waveform. Each bit is then modulated using a cosine carrier signal. When the input bit is 1, a higher amplitude carrier is transmitted; when the input bit is 0, a lower amplitude carrier is transmitted.

At the receiver, demodulation is performed using coherent detection. The received signal is multiplied by the carrier signal and integrated over the bit duration. The resulting value is compared with a threshold to decide whether the bit is 0 or 1.

The decision rule is:

- If the detected value is greater than threshold $\to$ bit = 1
- If the detected value is less than threshold $\to$ bit = 0

Finally, the recovered bits are reconstructed into a digital signal.

ASK is simple and easy to implement but is highly sensitive to noise because information is carried in the amplitude of the signal. Therefore, it is less robust compared to other modulation techniques like PSK or QAM.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# Input
x = np.array([1,0,1,0,1,0,1,0,1])
bp = 1e-6

print("Transmitted:", x)

# Digital Signal
bit = np.repeat(x, 100)
t1 = np.linspace(0, bp*len(x), len(bit))
```

```
# ASK Modulation
A1, A2 = 10, 5
f = 10/bp
t = np.linspace(0, bp, 100)

carrier = np.cos(2*np.pi*f*t)

m = np.concatenate([(A1 if i==1 else A2)*carrier for i in x])
t2 = np.linspace(0, bp*len(x), len(m))

# Demodulation
mn = []
for i in range(0, len(m), 100):
    seg = m[i:i+100]
    z = np.trapz(seg*carrier, t)
    mn.append(1 if (2*z/bp) > 7.5 else 0)

mn = np.array(mn)
print("Received:", mn)

# Reconstructed
bit_r = np.repeat(mn, 100)
t3 = np.linspace(0, bp*len(mn), len(bit_r))

# Plot
plt.figure(figsize=(10,8))

plt.subplot(3,1,1)
plt.plot(t1, bit)
plt.savefig("ask_input.png")

plt.subplot(3,1,2)
plt.plot(t2, m)
plt.savefig("ask_modulated.png")

plt.subplot(3,1,3)
plt.plot(t3, bit_r)
plt.savefig("ask_output.png")

plt.tight_layout()
plt.show()
```

Output

```
Transmitted: [1 0 1 0 1 0 1 0 1]
Received:    [1 0 1 0 1 0 1 0 1]
```

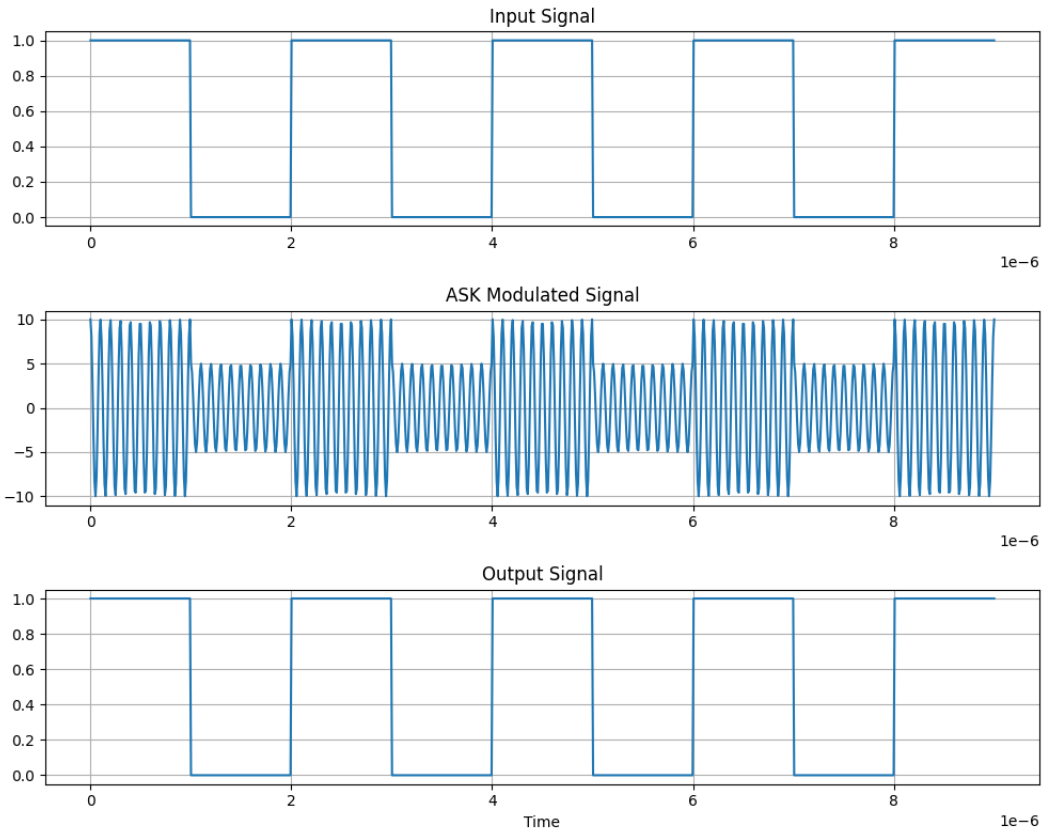

Figure 12: Reconstructed Output Signal

Result and Discussion

The experiment successfully implemented ASK modulation and demodulation. The modulated signal shows variation in amplitude corresponding to the input bits.

The demodulated output matches the transmitted signal, indicating correct detection using threshold-based decision making.

However, ASK is sensitive to noise, and any disturbance in amplitude may lead to incorrect detection.

Experiment No:08

Title

Implementation of Frequency Shift Keying (FSK) Modulation and Demodulation using Python.

Objective

- To understand the concept of Frequency Shift Keying (FSK).
- To generate FSK modulated signal from binary data.
- To perform coherent demodulation using frequency detection.
- To reconstruct the original signal from received signal.
- To analyze frequency variation in FSK signals.

Theory

Frequency Shift Keying (FSK) is a digital modulation technique in which the frequency of a carrier signal is varied according to the input binary data, while amplitude and phase remain constant. It is widely used in communication systems due to its better noise immunity compared to amplitude-based modulation schemes.

In binary FSK (BFSK), two different carrier frequencies are used to represent binary digits:

$$1 \to f_1 \quad \text{(high frequency)}, \qquad 0 \to f_2 \quad \text{(low frequency)}$$

The FSK signal can be expressed as:

$$s(t) = A\cos(2\pi f_i t)$$

where f_i is either f_1 or f_2 depending on the input bit, and A is the amplitude of the carrier signal.

In this experiment, a binary sequence is first converted into a digital waveform. Each bit is then transmitted using a cosine carrier with different frequencies depending on whether the bit is 1 or 0. Thus, frequency variation encodes the information.

At the receiver, demodulation is performed using correlation or coherent detection. The received signal is multiplied separately with both carrier frequencies and integrated over the bit duration. The outputs are compared to determine which frequency was present.

The decision rule is:

- If correlation with f_1 is higher $\to$ bit $= 1$
- Otherwise $\to$ bit $= 0$

FSK is more robust against noise compared to ASK because information is carried in frequency rather than amplitude. However, it requires more bandwidth and slightly higher complexity.

FSK is commonly used in applications such as wireless communication, radio transmission, and digital data communication systems.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# Input
x = np.array([1, 1, 0, 1, 0, 1])
bp = 1e-6

print("Binary information at transmitter")
print(x)

# Digital Signal
bit = np.repeat(x, 100)
t1 = np.linspace(0, bp*len(x), len(bit))
```

```
# FSK Modulation
A = 5
br = 1 / bp
f1 = br * 8
f2 = br * 2

t = np.linspace(0, bp, 100)

m = []
for i in x:
    if i == 1:
        y = A * np.cos(2*np.pi*f1*t)
    else:
        y = A * np.cos(2*np.pi*f2*t)
    m.extend(y)

m = np.array(m)
t2 = np.linspace(0, bp*len(x), len(m))

# Demodulation
mn = []
for i in range(0, len(m), 100):
    seg = m[i:i+100]

    y1 = np.cos(2*np.pi*f1*t)
    y2 = np.cos(2*np.pi*f2*t)

    z1 = np.trapz(seg*y1, t)
    z2 = np.trapz(seg*y2, t)

    if (2*z1/bp) > (A/2):
        mn.append(1)
    else:
        mn.append(0)

mn = np.array(mn)
print("Binary information at Receiver")
print(mn)

# Reconstructed
bit_r = np.repeat(mn, 100)
t3 = np.linspace(0, bp*len(mn), len(bit_r))

# Plot
plt.figure(figsize=(10,8))

plt.subplot(3,1,1)
plt.plot(t1, bit, linewidth=2.5)
plt.savefig("fsk_input.png")

plt.subplot(3,1,2)
plt.plot(t2, m)
plt.savefig("fsk_modulated.png")

plt.subplot(3,1,3)
plt.plot(t3, bit_r, linewidth=2.5)
plt.savefig("fsk_output.png")

plt.tight_layout()
plt.show()
```

Output

```
Binary information at transmitter: [1 1 0 1 0 1]
Binary information at Receiver:    [1 1 0 1 0 1]
```

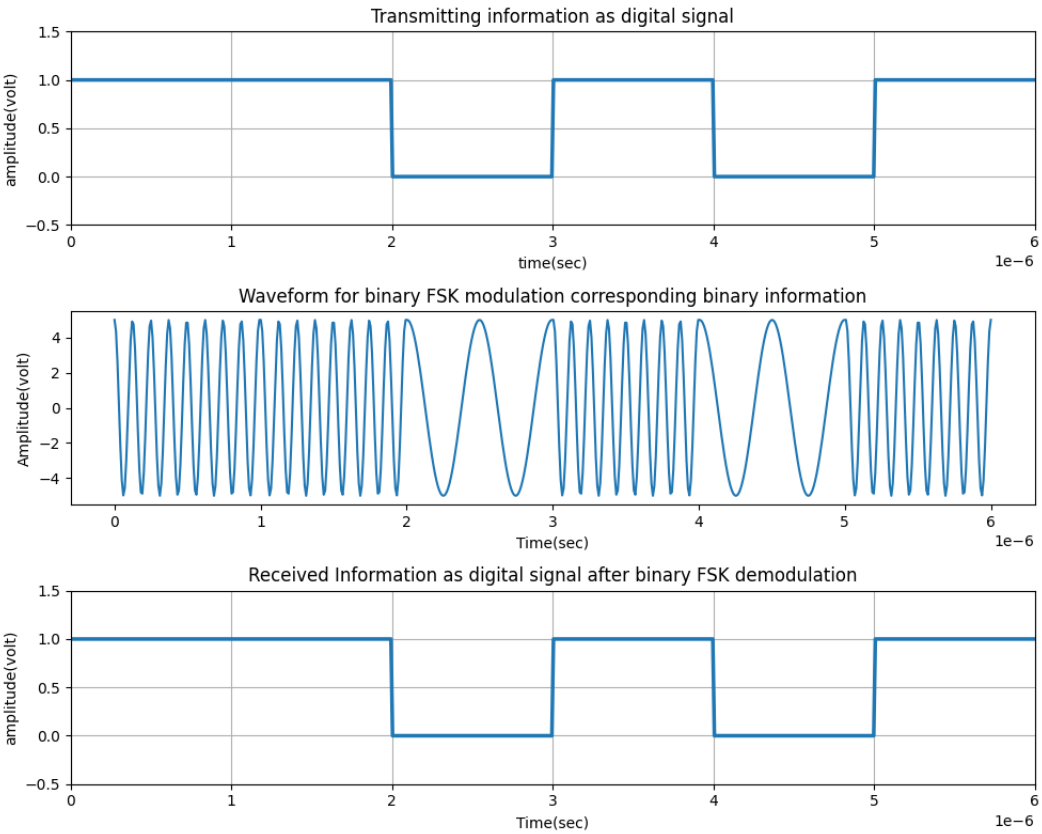

Figure 13: Input Digital Signal

Result and Discussion

The experiment successfully implemented FSK modulation and demodulation. The modulated signal clearly shows frequency variation corresponding to the input binary data.

The demodulated output matches the transmitted signal, confirming correct detection using frequency comparison.

It is observed that FSK provides better noise immunity compared to amplitude-based modulation techniques.

Conclusion

This experiment demonstrates the implementation of FSK modulation and demodulation. It confirms that FSK is a reliable modulation technique with improved resistance to noise, making it suitable for digital communication systems.

Experiment No:09

Title

Implementation of Phase Shift Keying (PSK) Modulation and Demodulation using Python.

Objective

- To understand the concept of Phase Shift Keying (PSK).
- To generate PSK modulated signal from binary data.
- To perform coherent demodulation using phase detection.
- To reconstruct the original signal from received signal.
- To analyze phase variation in PSK signals.

Theory

Phase Shift Keying (PSK) is a digital modulation technique in which the phase of a carrier signal is varied according to the input binary data, while the amplitude and frequency remain constant. It is widely used in communication systems due to its better noise immunity compared to amplitude-based modulation techniques.

In Binary Phase Shift Keying (BPSK), two different phases are used to represent binary digits:

$$1 \to 0^\circ \quad \text{(no phase shift)}, \qquad 0 \to 180^\circ \quad \text{(phase shift of } \pi \text{ radians)}$$

The PSK signal can be expressed as:

$$s(t) = A\cos(2\pi f_c t + \theta)$$

where θ is either 0 or π depending on the input bit.

In this experiment, a binary sequence is first converted into a digital waveform. Each bit is then transmitted using a cosine carrier signal. For bit 1, the carrier is transmitted without phase change, and for bit 0, the carrier is shifted by 180°.

At the receiver, demodulation is performed using coherent detection. The received signal is multiplied by a reference carrier signal and integrated over the bit duration. The sign of the resulting value determines the transmitted bit.

The decision rule is:

- If the output is positive $\to$ bit = 1
- If the output is negative $\to$ bit = 0

PSK is more robust than ASK because information is carried in phase rather than amplitude. It provides better performance in noisy environments and is widely used in modern digital communication systems.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# Input
x = np.array([1, 0, 1, 0, 1, 0, 1, 0, 1])
bp = 1e-6

print("Binary Information at transmitter")
print(x)

# Digital Signal
bit = np.repeat(x, 100)
t1 = np.linspace(bp/100, bp*len(x), len(bit))

# PSK Modulation
```

```
A = 5
br = 1 / bp
f = br * 2

t = np.linspace(bp/99, bp, 99)

m = []
for i in x:
    if i == 1:
        y = A * np.cos(2*np.pi*f*t)
    else:
        y = A * np.cos(2*np.pi*f*t + np.pi)
    m.extend(y)

m = np.array(m)
t2 = np.linspace(bp/99, bp*len(x), len(m))

# Demodulation
mn = []
ss = len(t)

for i in range(ss, len(m)+1, ss):
    seg = m[i-ss:i]

    carrier = np.cos(2*np.pi*f*t)
    z = np.trapz(seg * carrier, t)

    zz = round(2*z/bp)

    if zz > 0:
        mn.append(1)
    else:
        mn.append(0)

mn = np.array(mn)

print("Binary Information at Receiver After PSK Demodulation")
print(mn)

# Reconstructed
bit_r = np.repeat(mn, 100)
t3 = np.linspace(bp/100, bp*len(mn), len(bit_r))

# Plot
plt.figure(figsize=(10,8))

plt.subplot(3,1,1)
plt.plot(t1, bit, linewidth=2.5)
plt.savefig("psk_input.png")

plt.subplot(3,1,2)
plt.plot(t2, m)
plt.savefig("psk_modulated.png")

plt.subplot(3,1,3)
plt.plot(t3, bit_r, linewidth=2.5)
plt.savefig("psk_output.png")

plt.tight_layout()
plt.show()
```

Output

```
Binary Information at transmitter: [1 0 1 0 1 0 1 0 1]
Binary Information at Receiver:    [1 0 1 0 1 0 1 0 1]
```

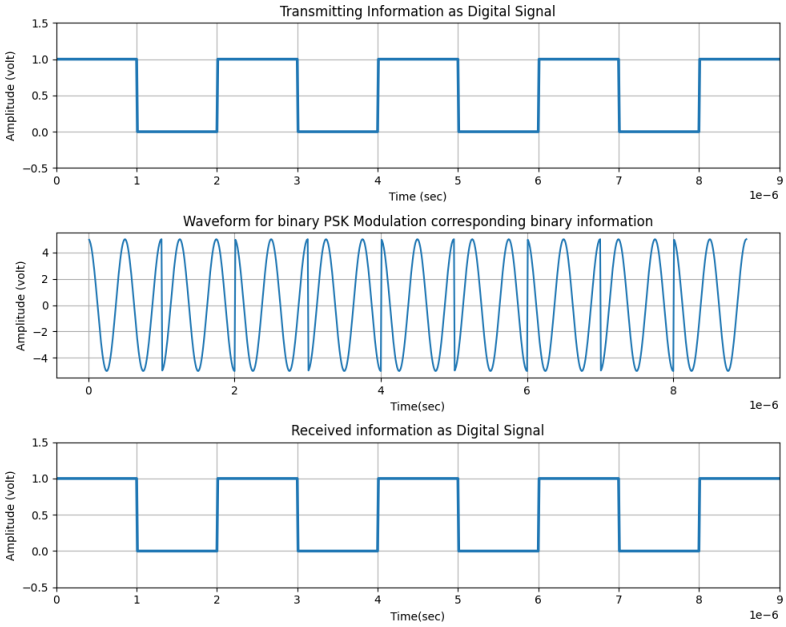

Figure 14: Input Digital Signal

Result and Discussion

The experiment successfully implemented PSK modulation and demodulation. The modulated signal shows phase reversals corresponding to the input binary data.

The demodulated output matches the transmitted signal, confirming correct detection using phase comparison.

Experiment No:10

Title

Generation of Pulse Amplitude Modulation (PAM) using Sampling Pulse and Message Signal in Python.

Objective

- To understand the concept of Pulse Amplitude Modulation (PAM).
- To generate a sampling pulse signal.
- To generate a message (analog) signal.
- To obtain PAM signal using sampling technique.
- To analyze relationship between sampling pulse and message signal.

Theory

Pulse Amplitude Modulation (PAM) is a modulation technique in which the amplitude of regularly spaced pulses is varied in accordance with the instantaneous value of a message signal. It is one of the fundamental techniques used in analog and digital communication systems.

In PAM, a continuous-time message signal is sampled using a periodic pulse train. The amplitude of each pulse is proportional to the amplitude of the message signal at that instant. Thus, the information of the message signal is carried in the amplitude of the pulses.

The PAM signal can be mathematically expressed as:

$$s(t) = m(t) \cdot p(t)$$

where $m(t)$ is the message signal and $p(t)$ is the periodic pulse train.

In this experiment, a square wave is used as the sampling pulse. The duty cycle of the square wave determines the width of each pulse. Only during the active (ON) period of the pulse, the message signal is transmitted.

The message signal is chosen as a sinusoidal signal. The PAM signal is generated by sampling the message signal at regular intervals defined by the sampling frequency.

The sampling process follows the Nyquist sampling theorem, which states that the sampling frequency must be at least twice the highest frequency present in the message signal to avoid aliasing.

The resulting PAM signal consists of discrete pulses whose amplitudes follow the shape of the original message signal. This allows reconstruction of the original signal at the receiver using appropriate filtering techniques.

PAM is widely used in communication systems such as time-division multiplexing (TDM), digital telephony, and signal processing applications.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# Parameters
fc = 20
fm = 2
fs = 1000
t = 1

n = np.arange(0, t, 1/fs)

duty = 20

# Square pulse
s = signal.square(2*np.pi*fc*n, duty/100)
s[s < 0] = 0
```

```
# Message signal
m = np.sin(2*np.pi*fm*(n-1))

# PAM generation
period_samp = int(len(n)/fc)
ind = np.arange(0, len(n), period_samp)
on_samp = int(np.ceil(period_samp * duty / 100))

pam = np.zeros(len(n))

for i in ind:
    pam[i:i+on_samp] = m[i]

# Plot
plt.figure(figsize=(10,8))

plt.subplot(3,1,1)
plt.plot(n, s)
plt.savefig("pam_pulse.png")

plt.subplot(3,1,2)
plt.plot(n, m)
plt.savefig("pam_message.png")

plt.subplot(3,1,3)
plt.plot(n, pam)
plt.savefig("pam_output.png")

plt.tight_layout()
plt.show()
```

Output

```
PAM signal generated successfully using sampling pulse.
```

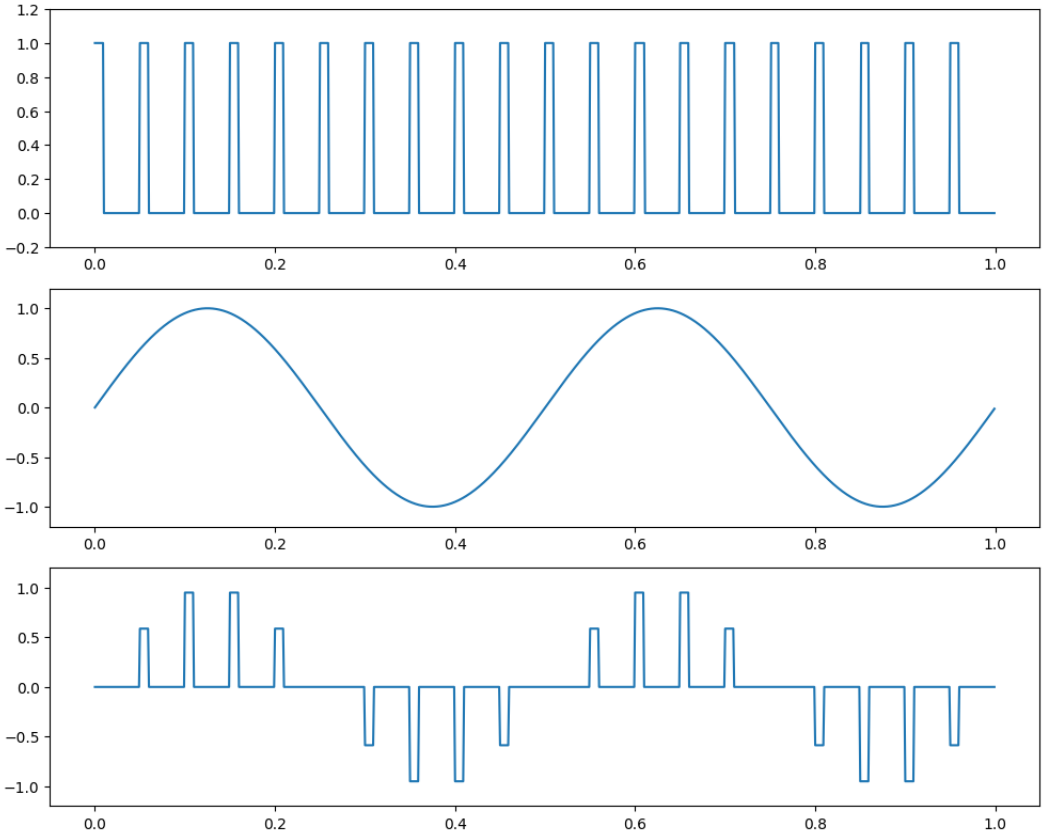

Figure 15: Sampling Pulse Signal

Result and Discussion

The experiment successfully generated a PAM signal using a sampling pulse and a message signal. The PAM waveform shows that the amplitude of pulses follows the shape of the message signal.

The sampling pulse controls when the message signal is transmitted. Only during the active period of the pulse, the message signal is sampled.

This demonstrates how PAM effectively converts a continuous signal into a pulse-based signal for transmission.

Experiment No:11

Title

Generation of Unipolar Non-Return-to-Zero (NRZ) Line Coding using Python.

Objective

- To understand the concept of line coding.
- To implement Unipolar NRZ signaling.
- To represent binary data in waveform form.
- To analyze characteristics of NRZ signaling.
- To visualize digital signal transmission.

Theory

Line coding is a technique used in digital communication systems to convert binary data into electrical signals suitable for transmission over a communication channel. It plays an important role in data transmission, synchronization, and error detection.

Unipolar Non-Return-to-Zero (NRZ) is one of the simplest line coding techniques. In this method, only one polarity (positive voltage) is used to represent binary data.

The mapping rule for Unipolar NRZ is:

$$1 \rightarrow +V, \qquad 0 \rightarrow 0$$

where $+V$ represents a constant positive voltage level.

In NRZ signaling, the signal does not return to zero between bits. That is, the signal level remains constant throughout the bit duration. If consecutive bits are the same, the signal remains unchanged.

In this experiment, a binary sequence is first generated. Each bit is then mapped to its corresponding voltage level using Unipolar NRZ rules. The signal is shaped such that each bit occupies a fixed time interval.

The resulting waveform is a step-like signal where the amplitude changes only when the bit value changes. This type of signaling is easy to implement but has some limitations.

One major drawback of Unipolar NRZ is the presence of a DC component, which makes it unsuitable for certain communication channels. It also lacks synchronization capability when long sequences of zeros occur.

Despite its limitations, Unipolar NRZ is useful for understanding basic digital signal representation.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.array([0,1,1,0,0,0,1,0,1,0])
print("Binary Data:", n)

# Mapping
nn = []
for m in range(N):
    if n[m] == 1:
        nn.append(1)
    else:
        nn.append(0)

# Signal shaping
i = 0
t = np.arange(0, N, 0.01)
y = np.zeros(len(t))
```

```
for j in range(len(t)):
    if t[j] <= i+1:
        y[j] = nn[i]
    else:
        y[j] = nn[i]
        i += 1

# Plot
plt.step(t, y, where='post', linewidth=2)

plt.axis([0, N, -1.5, 1.5])
plt.xticks(range(0, N+1))
plt.grid(True)
plt.title("Unipolar NZR Signaling")

plt.savefig("nrz_signal.png")
plt.show()
```

Output

```
Binary Data: [0 1 1 0 0 0 1 0 1 0]
```

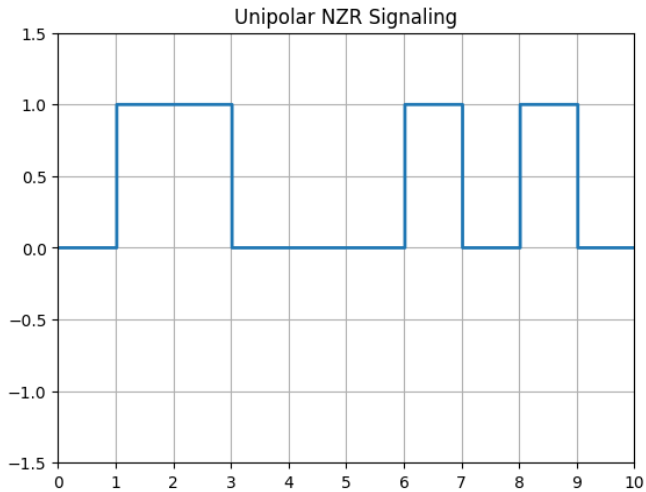

Figure 16: Unipolar NRZ Line Coding Signal

Result and Discussion

The experiment successfully generated a Unipolar NRZ signal from binary input data. The waveform shows constant amplitude levels corresponding to binary values.

The signal remains at a positive level for binary 1 and zero level for binary 0, without returning to zero between bits.

This demonstrates how digital data can be transmitted using simple line coding techniques.

Experiment No:12

Title

Generation of Polar Non-Return-to-Zero (NRZ) Line Coding using Python.

Objective

- To understand the concept of Polar NRZ line coding.
- To convert binary data into bipolar voltage levels.
- To visualize the digital waveform representation.
- To analyze advantages over unipolar signaling.
- To study signal characteristics in time domain.

Theory

Line coding is an essential technique in digital communication systems used to convert binary data into electrical signals for transmission. Among various line coding techniques, Polar Non-Return-to-Zero (NRZ) is widely used due to its improved performance over unipolar signaling.

In Polar NRZ, binary data is represented using two opposite voltage levels:

$$1 \rightarrow +V, \qquad 0 \rightarrow -V$$

Unlike unipolar NRZ, Polar NRZ uses both positive and negative voltages, which eliminates the DC component and improves signal quality.

In this technique, the signal does not return to zero between bits. Instead, it remains at a constant level throughout the bit duration. When consecutive bits are identical, the signal level remains unchanged.

In this experiment, a binary sequence is generated and mapped into corresponding voltage levels using Polar NRZ rules. The signal is then shaped over time such that each bit occupies a fixed interval.

The resulting waveform is a step-like signal with two voltage levels, positive and negative, corresponding to binary values. This improves synchronization and reduces baseline wandering compared to unipolar signaling.

Polar NRZ provides better performance in communication systems because it reduces power loss and improves noise immunity. However, it still has limitations such as lack of self-synchronization in long sequences of identical bits.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt
N = 10
n = np.array([0,1,0,1,0,1,0,1,1,1])
print("Binary Data:", n)
# Mapping
nn = []
for m in range(N):
    if n[m] == 1:
        nn.append(1)
    else:
        nn.append(-1)
# Signal shaping
i = 0
t = np.arange(0, N, 0.01)
y = np.zeros(len(t))

for j in range(len(t)):
    if t[j] <= i+1:
        y[j] = nn[i]
```

```
    else:
        y[j] = nn[i]
        i += 1
plt.step(t, y, where='post', linewidth=2)
plt.axis([0, N, -1.5, 1.5])
plt.xticks(range(0, N+1))
plt.grid(True)
plt.title("Polar NZR Signaling")
plt.savefig("polar_nrz.png")
plt.show()
```

Output

```
Binary Data: [0 1 0 1 0 1 0 1 1 1]
```

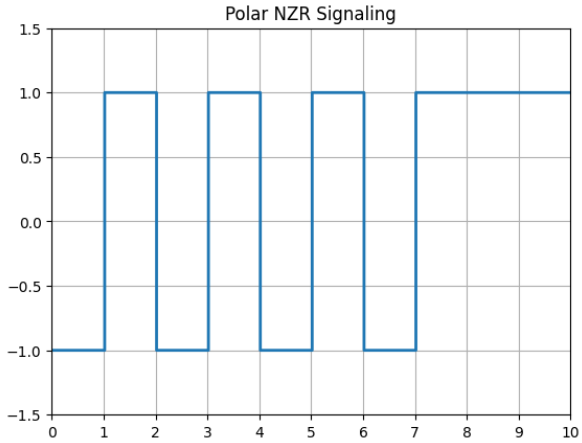

Figure 17: Polar NRZ Line Coding Signal

Result and Discussion

The experiment successfully generated a Polar NRZ signal from binary input data. The waveform shows two voltage levels corresponding to binary values.

Unlike unipolar NRZ, the signal alternates between positive and negative levels, which reduces DC component and improves transmission efficiency.

This demonstrates how Polar NRZ provides better performance compared to unipolar signaling.

Experiment No:13

Title

Generation of Unipolar Return-to-Zero (RZ) Line Coding using Python.

Objective

- To understand the concept of Return-to-Zero (RZ) line coding.
- To implement Unipolar RZ signaling.
- To represent binary data using pulse signals.
- To analyze signal behavior within bit duration.
- To compare RZ with NRZ signaling techniques.

Theory

Line coding is a technique used in digital communication to convert binary data into electrical signals for transmission over a communication channel. Among different types of line coding, Return-to-Zero (RZ) is an important method where the signal returns to zero within each bit duration.

In Unipolar RZ signaling, binary data is represented using only positive voltage levels and zero. The mapping rule is:

$$1 \rightarrow +V \text{ (for half of bit duration)}, \qquad 0 \rightarrow 0$$

Unlike Non-Return-to-Zero (NRZ), the signal in RZ returns to zero before the end of each bit period. This means that even if consecutive bits are the same, the signal drops to zero between bits.

In this experiment, a binary sequence is generated and mapped into corresponding voltage levels. Each bit duration is divided into two halves:

- First half: signal carries the bit value
- Second half: signal returns to zero

This produces a pulse-like waveform where each bit is represented by a pulse followed by zero level.

The advantage of Unipolar RZ is that it provides better synchronization because the signal returns to zero frequently, allowing easier clock recovery at the receiver. However, it requires more bandwidth compared to NRZ because of additional transitions.

Another limitation is that it still contains a DC component due to the use of only positive voltages.

Thus, Unipolar RZ is useful for understanding pulse-based signaling, though it is less efficient compared to other advanced line coding techniques.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.array([1,1,0,1,1,0,0,1,1,1])
print("Binary Data:", n)

i = 0
a = 0
b = 0.5
t = np.arange(0, N, 0.01)
y = np.zeros(len(t))

for j in range(len(t)):
    if t[j] >= a and t[j] <= b:
        y[j] = n[i]
```

```
    elif t[j] > b and t[j] <= i+1:
        y[j] = 0
    else:
        i += 1
        a += 1
        b += 1

plt.step(t, y, where='post', linewidth=2)

plt.axis([0, N, -1.5, 1.5])
plt.xticks(range(0, N+1))
plt.grid(True)
plt.title('Unipolar return-to-zero (RZ) signaling')

plt.savefig("unipolar_rz.png")
plt.show()
```

Output

```
Binary Data: [1 1 0 1 1 0 0 1 1 1]
```

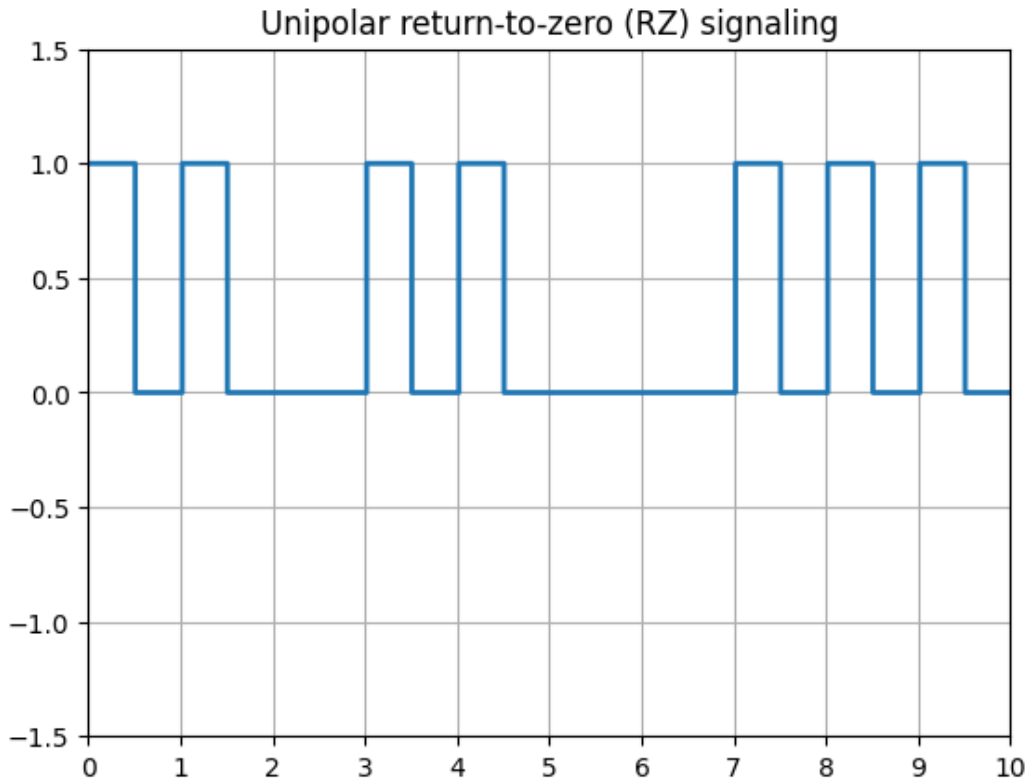

Figure 18: Unipolar RZ Line Coding Signal

Result and Discussion

The experiment successfully generated a Unipolar RZ signal from binary input data. The waveform shows that each bit returns to zero within its duration.

This creates more transitions compared to NRZ, which improves synchronization. However, it also increases bandwidth requirements.

The experiment demonstrates the trade-off between synchronization and bandwidth in digital communication systems.

Experiment No:14

Title

Generation of Bipolar Return-to-Zero (RZ) Line Coding using Python.

Objective

- To understand the concept of Bipolar RZ (AMI) line coding.
- To implement alternate mark inversion (AMI) technique.
- To represent binary data using positive and negative pulses.
- To analyze advantages over unipolar and polar signaling.
- To study signal characteristics in time domain.

Theory

Line coding is used in digital communication systems to convert binary data into electrical signals for transmission. Bipolar Return-to-Zero (RZ), also known as Alternate Mark Inversion (AMI), is an advanced line coding technique that improves upon unipolar and polar schemes.

In Bipolar RZ signaling, binary data is represented as:

$$1 \to +V,\ -V \text{ (alternating)}, \qquad 0 \to 0$$

This means that every binary 1 is transmitted as a pulse whose polarity alternates between positive and negative. Binary 0 is represented by zero voltage.

In addition, the signal returns to zero within each bit duration, similar to RZ signaling. Thus, each bit period consists of:

- First half: signal carries +V or -V (for bit 1) or 0 (for bit 0)
- Second half: signal returns to zero

The key feature of Bipolar RZ is the alternating polarity of successive 1s. This eliminates the DC component and improves signal balance, making it suitable for long-distance transmission.

In this experiment, a binary sequence is first generated. For each bit:

- If the bit is 1, a pulse is generated with alternating polarity
- If the bit is 0, the signal remains at zero

The waveform is then shaped over time with return-to-zero property.

The advantages of Bipolar RZ include:

- No DC component
- Better synchronization due to frequent transitions
- Error detection capability (violation of polarity rule)

However, it requires more bandwidth than NRZ due to additional transitions.

Bipolar RZ is widely used in communication systems such as telephony and data transmission where reliable and balanced signaling is required.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.array([0,0,1,1,0,0,0,1,1,1])
print("Binary Data:", n)
```

```
# Bipolar mapping
nn = []
f = 1
for m in range(N):
    if n[m] == 1:
        nn.append(f)
        f *= -1
    else:
        nn.append(0)

i = 0
a = 0
b = 0.5
t = np.arange(0, N, 0.01)
y = np.zeros(len(t))

for j in range(len(t)):
    if t[j] >= a and t[j] <= b:
        y[j] = nn[i]
    elif t[j] > b and t[j] <= i+1:
        y[j] = 0
    else:
        i += 1
        a += 1
        b += 1

plt.step(t, y, where='post', linewidth=2)

plt.axis([0, N, -1.5, 1.5])
plt.xticks(range(0, N+1))
plt.grid(True)
plt.title('Bipolar return-to-zero (BRZ) signaling')

plt.savefig("bipolar_rz.png")
plt.show()
```

Output

```
Binary Data: [0 0 1 1 0 0 0 1 1 1]
```

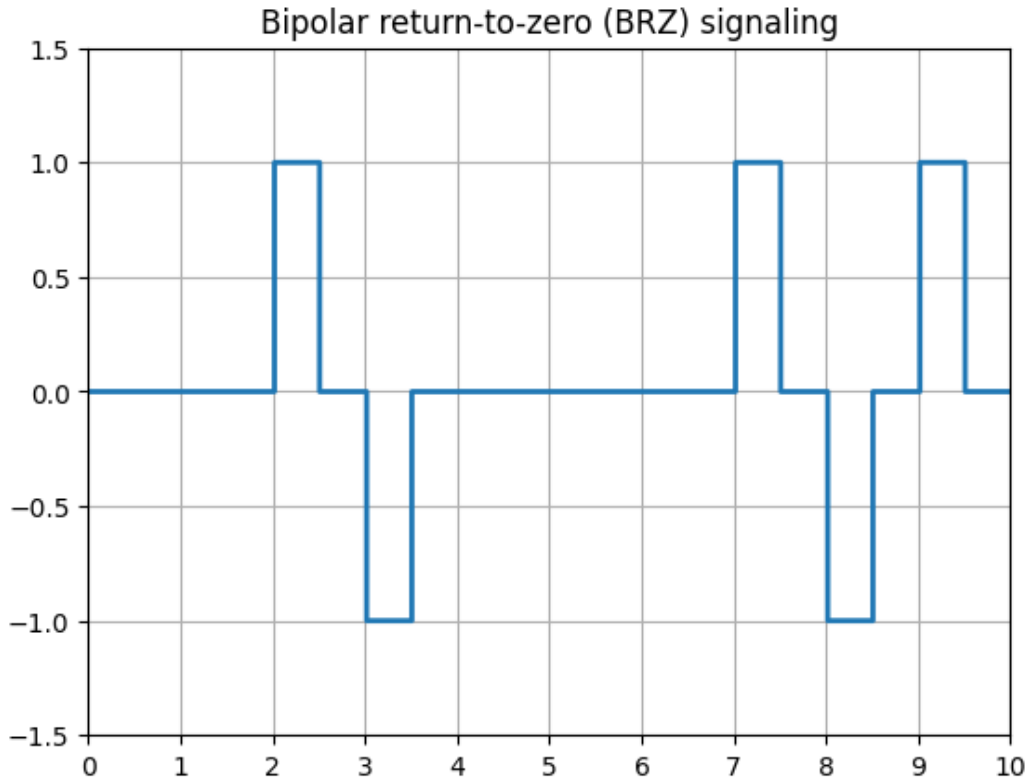

Figure 19: Bipolar RZ (AMI) Line Coding Signal

Result and Discussion

The experiment successfully generated a Bipolar RZ signal using AMI coding. The waveform shows alternating positive and negative pulses for binary 1 and zero level for binary 0.

The signal returns to zero within each bit duration, providing better synchronization. The alternating polarity eliminates the DC component and improves transmission quality.

This demonstrates that Bipolar RZ is more efficient and reliable compared to unipolar and polar signaling.

Experiment No:15

Title

Generation of Manchester Line Coding using Python.

Objective

- To understand the concept of Manchester line coding.
- To encode binary data using Manchester scheme.
- To analyze signal transitions within bit duration.
- To study synchronization properties of Manchester coding.
- To visualize digital waveform representation.

Theory

Manchester coding is a type of digital line coding technique in which each bit of data is represented by a transition in the signal. It combines both clock and data information into a single signal, making it highly reliable for synchronization.

In Manchester coding, each bit period is divided into two equal halves, and a transition occurs at the middle of each bit. The encoding rule is:

$$1 \rightarrow \text{High to Low transition}, \qquad 0 \rightarrow \text{Low to High transition}$$

Thus, each bit is represented by two signal levels:

- Binary 1: positive level followed by negative level
- Binary 0: negative level followed by positive level

This ensures that there is always a transition in the middle of every bit period, which helps in clock recovery at the receiver.

In this experiment, a binary sequence is first generated. Each bit is then mapped into two signal levels according to Manchester coding rules. The signal is shaped such that each half-bit interval carries one level.

The resulting waveform alternates between positive and negative levels within each bit duration. This eliminates the DC component and improves synchronization.

The advantages of Manchester coding include:

- Self-synchronizing (easy clock recovery)
- No DC component
- Reliable data transmission

However, it requires double the bandwidth compared to NRZ because each bit is represented by two signal transitions.

Manchester coding is widely used in communication systems such as Ethernet (LAN networks), RFID systems, and digital communication protocols.

Code (Python)

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.array([0,0,1,0,0,0,0,0,0,1])
print("Binary Data:", n)

# Mapping
nnn = []
```

```
for m in range(N):
    if n[m] == 1:
        nn = [1, -1]
    else:
        nn = [-1, 1]
    nnn.extend(nn)

i = 0
l = 0.5
t = np.arange(0, N, 0.01)
y = np.zeros(len(t))

for j in range(len(t)):
    if t[j] <= l:
        y[j] = nnn[i]
    else:
        y[j] = nnn[i]
        i += 1
        l += 0.5

plt.step(t, y, where='post', linewidth=2)

plt.axis([0, N, -1.5, 1.5])
plt.xticks(range(0, N+1))
plt.grid(True)
plt.title('Manchester Coding')

plt.savefig("manchester.png")
plt.show()
```

Output

```
Binary Data: [0 0 1 0 0 0 0 0 0 1]
```

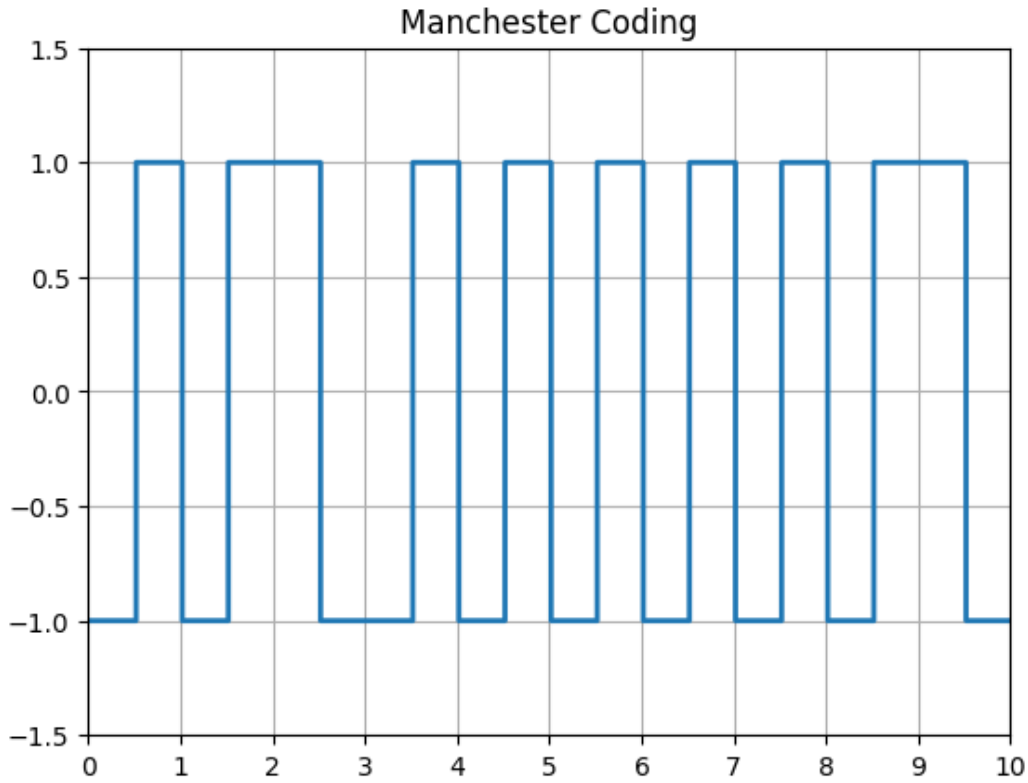

Figure 20: Manchester Line Coding Signal

Result and Discussion

The experiment successfully generated a Manchester coded signal from binary input data. The waveform shows a transition at the middle of each bit period.

This ensures proper synchronization and eliminates DC component. The signal clearly represents binary data through transitions rather than constant levels.

The experiment highlights the advantage of Manchester coding in maintaining timing accuracy.